

پروژه نهایی سیگنال ها و سیستم ها

کسرا نوربخش ۸۱۰۱۰۰۲۳۰

من در 2 فاز ابتدایی این پروژه از ChatGPT استفاده کرده ام.

لینک چت برای گام اول (فقط تا قبل از وقتی که نوشته ام ... now in this phase we...)
<https://chatgpt.com/share/677beea0-5d68-800d-8730-3165a1369775>

لینک چت برای گام دوم (فایل این کد موجود در part2NotGood.py)
<https://chatgpt.com/share/677cd71b-c6c0-800d-a506-763c0374ab41>

که صدای تولید شده اش خوب نبود و به سراغ یک تلاش دیگر رفتم
<https://chatgpt.com/share/677cf24a-9420-800d-b08e-26b04caac75c>

مقدمه

این پروژه را با زبان python انجام دادیم و از کتابخانه های numpy (برای محاسبات مانند محاسبه سینوس و گرفتن fft)، scipy (برای signal processing و نوشتن در فایل)، matplotlib (برای کشیدن نمودار ها) و os (برای کار با فایل ها) استفاده کردیم.

```
PS D:\semester 7\SS\Project> pip install numpy scipy
Requirement already satisfied: numpy in c:\users\kasra\AppData\Local\Programs\Python\Python313\lib\site-packages (2.1.3)
Collecting scipy
  Downloading scipy-1.15.0-cp313-cp313-win_amd64.whl.metadata (60 kB)
  Downloading scipy-1.15.0-cp313-cp313-win_amd64.whl (43.6 MB)
  43.6/43.6 MB 1.3 MB/s eta 0:00:00
Installing collected packages: scipy
Successfully installed scipy-1.15.0
```

عکس 1: نحوه نصب کتابخانه های numpy و scipy که برای اجرای پروژه مورد نیاز است (با فرض این که پیش تر، pip نصب شده است و نصب مابقی کتابخانه ها هم به همین صورت است)

گام اول

در گام اول با استفاده از جدول فرکانس های داده شده برای هر نت و اکتاو های مختلف هر نت، به همراه فایل notes.m که شامل نت ها و اکتاو ها و زمان هر نت موسیقی فیلم Harry Potter بود، توانستیم موسیقی را بسازیم و در فایل noteHarryShutter.wav ذخیره کنیم.

توضیح کد این گام:

در آرایه noteOctave4Freqs فرکانس های مربوط به اکتاو 4 هر نت مطابق با فرکانس های داده شده جدول، ذخیره شد. در اینجا از اکتاو 4 استفاده کردیم چون مرجع موسیقی غربی است و شامل middle C یا همان C4 است که tuning pitch می باشد.

با استفاده از تابع get_frequency، بسته به آن که نت و اکتاو ما چیست، فرکانس آن را با فرمول $f_{base} \times 2^{(o-4)}$ محاسبه کردیم چون در موسیقی وقتی که یک اکتاو بالا می رویم، فرکانس پایه ما حدوداً 2 برابر می شود (که این موضوع در جدول داده شده دیده شد).

سپس با استفاده از تابع synthesize_note، برای هر نت با توجه به فرکانسی که پیش تر محاسبه شد و مدت زمانی که در آرایه اولیه موجود بود و فرکانس سمپلینگ که در پروژه گفته شده بود 44100، موج سینوسی ساخته شد.

با استفاده از تابع parse_note، برای هر عضوی از آرایه اولیه، نام نت و مدت زمانش و اکتاوش استخراج شد. در نهایت هم با استفاده از تابع generate_signal کل موسیقی را ساختیم، یعنی به ازای هر نوت و اکتاو و مدت زمانی (در واقع هر عضوی از آرایه اولیه داده شده)، این کار را انجام دادیم:

```
note, octave, duration = parse_note(noteStr)
freq = get_frequency(note, octave)
tone = synthesize_note(freq, duration, sampleRate)
song = np.concatenate((song, tone, silence))
```

در آخر هم که آرایه نت ها را از فایل کپی کردیم و آوردیم و سیگنال را ساختیم و ذخیره کردیم و با استفاده از این خط کد:

```
song = np.int16(song / np.max(np.abs(song)) * 32767)
```

خروجی را نرمالایز کردیم تا بین -1 تا 1 باشد و سپس در عدد 32767 ضرب کردیم که در ناحیه مجاز فرمت wav. که می خواهیم ذخیره کنیم باشد.

گام دوم

در گام دوم با استفاده از نت های اکتاو 5، و استخراج هارمونیک های مختلف آن ها توانستیم نت های صاف تر و ملایم تری از گام اول را بسازیم و با استفاده از آن ها آهنگ بخش قبل را نواختیم و در فایل `noteHarryOptimize.wav` ذخیره کردیم. همچنین در فولدر `notesOptimized` نوت های جدید موجود اند.

توضیح کد این گام:

در ابتدا که با استفاده از تابع `load_wav` نت های اولیه را که در فولدر `notes` موجود اند، خواندیم. با تابع `save_optimized_note_to_wav` می توانیم نت های ساخته شده را در فولدر `notesOptimized` ذخیره کنیم. تابع `plot_fft` هم که برای کشیدن تبدیل های فوریه به کار می رود که پیاده سازی اش مشخص است.

تابع `get_harmonics` به نوعی تابع اصلی ماست که در آن ابتدا از هر نوت تبدیل فوریه می گیریم و فرکانس هایش را استخراج می کنیم، سپس فقط فرکانس های مثبت آن را نگه می داریم یعنی:

```
positiveFreqs = freqs[:N//2]
positiveFft = np.abs(fftResult[:N//2])
```

با گرفتن `argmax` برای روی فرکانس ها، فرکانس اوج یا همان `peak` را بدست می آوریم که بسیار کاربردی است. برای به دست آوردن هارمونی های مختلف، مضارب فرکانس `peak` را بدست می آوریم (این نکته را هم می شد از جدول به دست آورد هم اینکه در شکل تبدیل فوریه که رسم می کنیم به این موضوع پی می بریم). سپس با این خطوط کد، `harmonicFreq` را که از روی مضارب فرکانس پیک ساخته بودیم، مقایسه می کنیم با فرکانس های موجود در تبدیل فوریه و هارمونی ها را استخراج می کنیم.

با تابع `generate_optimized_note` از داده هایی که پیش تر به دست آوردیم، به سراغ ساختن نت های جدید می رویم که متشکل از جمع سینوسی ها با فرکانس های جدید اند که در انتها دمپ هم می شوند (توسط e^{-2t}). اثر دمپینگ هم بررسی گردید که اگر نباشد نت ایجاد شده همینجور نواخته می شود و بلند تر می شود که خوب نیست و اگر آن را دمپ بکنیم بیشتر به صدای پیانو شبیه می شود. در این تابع ما این خطوط کد را داریم:

```
optimizedSignal = amplitude * np.sin(2 * np.pi * fundamentalFreq * t)

for harmonic in harmonics:
    freq, amp = harmonic
    harmonicSignal = (amplitude * amp) * np.sin(2 * np.pi * freq * t)
    optimizedSignal += harmonicSignal
```

که ابتدا خود نت اصلی را می سازیم (مشابه گام اول) و سپس هارمونی ها را با آن جمع می کنیم. با هم به مانند گام اول خط:

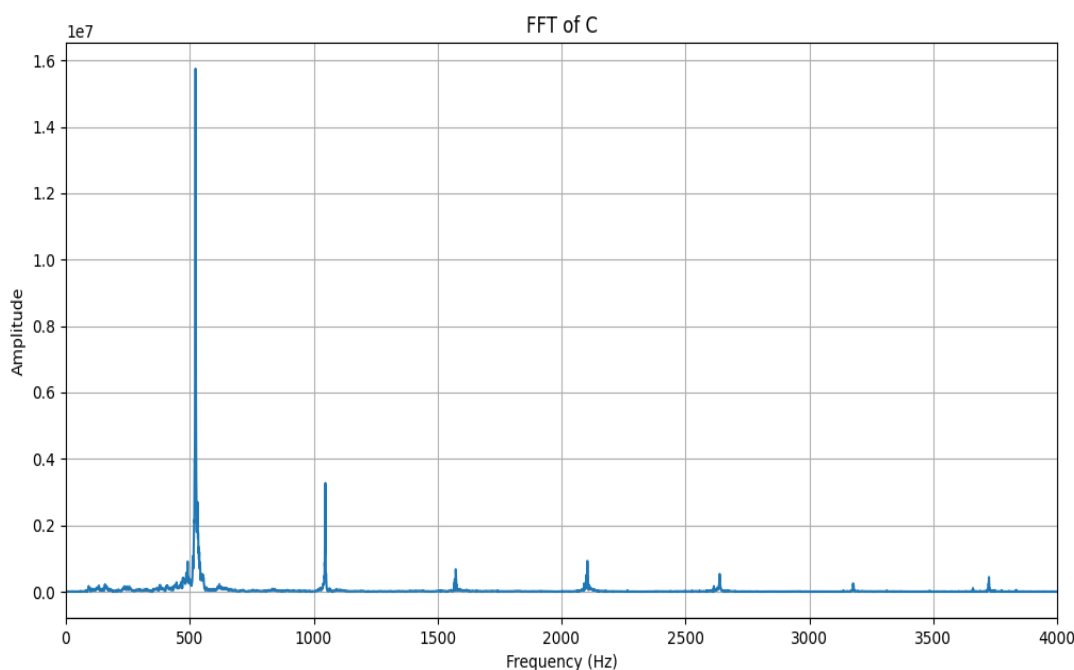
```
optimizedSignal = np.int16(optimizedSignal / np.max(np.abs(optimizedSignal)) * 32767)
```

را داریم که در فرمت مورد نظر ما یعنی `wav`. ذخیره شود.

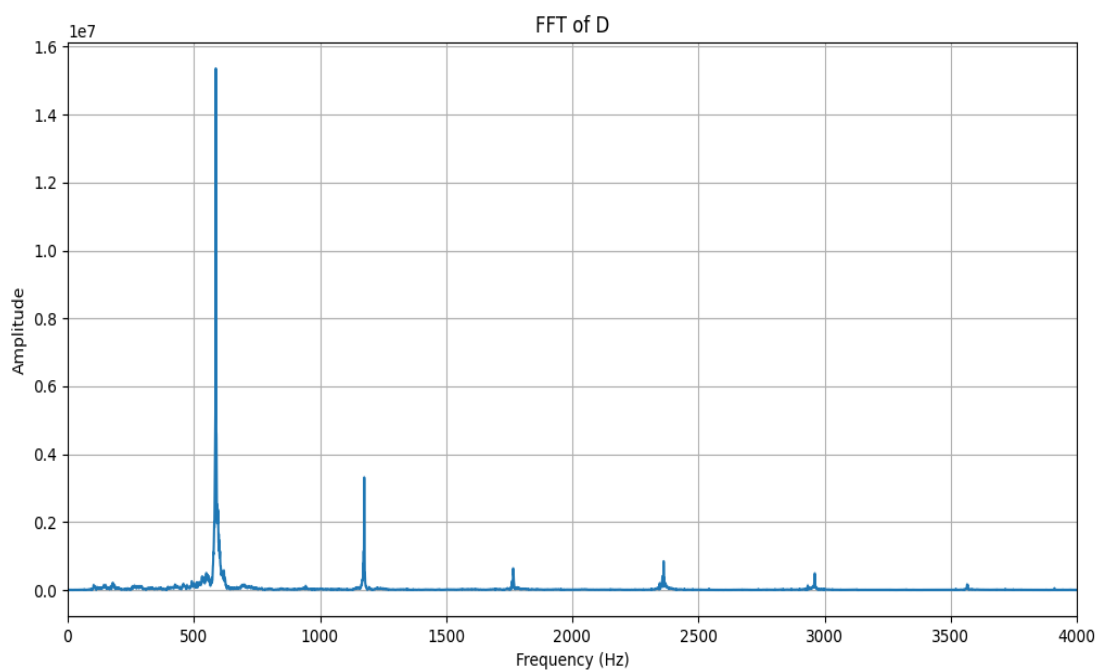
با استفاده از تابع `create_song` همان نوت های موجود در گام اول را، این بار با نوت های به دست آمده و ساخته شده خودمان تولید می کنیم. تابع `process_notes` از تمامی توابع پیش تر تعریف شده استفاده می کند تا نوت های جدید ما را بسازد. با این دو خط موجود در آن:

```
harmonicValues = [harmonic[1] for harmonic in harmonics]
harmonicsData.append([noteName, fundamentalFreq] + harmonicValues)
```

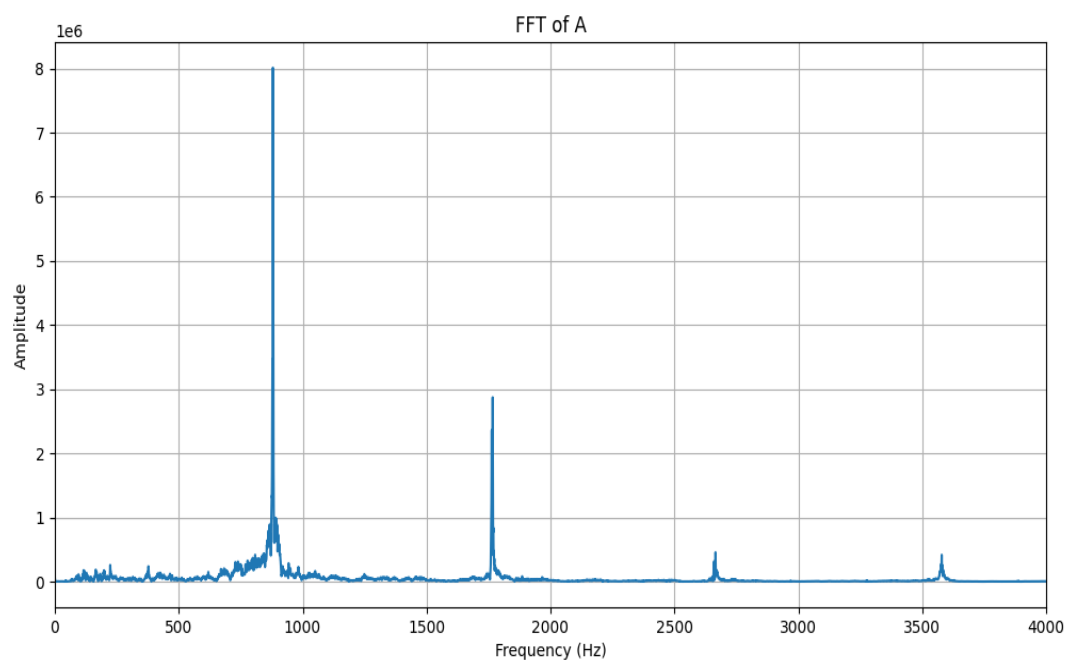
داده های مورد نیاز برای فایل اکسل را نگه می داریم. در انتهای این تابع هم، نمودار تبدیل فوریه را برای سه نت (A, C, D) رسم می کنیم (مشاهده می شود که هارمونی های مختلف دارند تقریباً به صورت مضربی از فرکانس پایه تکرار می شوند):



عکس 2: نمودار تبدیل فوریه نت C (فرکانس پایه 510، پیک هایی حدود 1100 و 1600 و 2200 و ...)



عکس 3: نمودار تبدیل فوریه نت D



عکس 4: نمودار تبدیل فوریه نت A

گام امتیازی

در گام امتیازی با استفاده از فایل noteHarryShutter.wav که در گام اول خروجی گرفته بودیم، توانستیم نت ها و اکتاو ها و زمان هر نت موسیقی فیلم Harry Potter را با بهره گیری از تبدیل فوریه و کانولوشن، استخراج کنیم و در نهایت آن را با محتویات notes.m مقایسه کنیم که عیناً برابر بودند.

توضیح کد این گام:

در آرایه noteOctave0Freqs فرکانس های مربوط به اکتاو 0 هر نت مطابق با فرکانس های داده شده جدول، ذخیره شد. با باز استفاده کردن از تابع load_wav که در گام دوم نوشته بودیم، فایل صوتی را لود کردیم.

با استفاده از تابع detect_song_segment بخش هایی که نتی نواخته می شود و یا اینکه سکوتی است را تشخیص می دهیم. ما در این تابع پس از اینکه دامنه یک سیگنال را استخراج کردیم، با convolution آن را smooth می کنیم و اگر 0 بود می فهمیم که سکوت بوده است. با بهره گیری از silenceMask بخش هایی که سکوت داریم را دور میریزیم و در نهایت زمان شروع و پایان هر بخش به همراه اندیس آن را باز میگردانیم که بعد تر در خروجی دادن هم به کارمان می آید.

با بکارگیری تابع extract_notes، در بخش هایی که پیش تر آن ها را بدست آوردیم، فرکانس را استخراج می کنیم. به مانند بخش دوم در اینجا هم تبدیل فوریه می گیریم و هم فرکانس های تبدیل فوریه را بدست می آوریم و آن فرکانسی که در پیک است را به عنوان فرکانس آن نوت استخراج می کنیم (که به صورت عملی در نمودار تبدیل فوریه نت ها در بخش 2 دیده بودیم).

در نهایت هم با تابع find_note_freq فرکانس هایی را که بدست آوردیم، با فرکانس های معروف نت ها (طبق جدول) مقایسه می کند و آن را که کمترین اختلاف فرکانسی را دارد به عنوان نتیجه ذخیره می کند. همچنین این مقایسه را برای اکتاو های مختلف انجام می دهیم.

روندی که صورت می گیرد:

```
sampleRate, data = load_wav("noteHarryShutter.wav")
songSegments, segments = detect_song_segments(data, sampleRate)
notesFrequencies = extract_notes(segments, data, sampleRate)
notes = find_note_freq(notesFrequencies, noteOctave0Freqs)
durations = [round(end - start, 1) for start, end in songSegments]
```

در آخر، آرایه نت ها و اکتاو ها را چاپ می کنیم و اگر دقیقاً مشابه با آرایه ابتدایی بود، می نویسیم که برابر بودند:

```
PS D:\semester 7\SS\Project> python part3.py
['B 4 0.3', 'E 5 0.6', 'G 5 0.2', 'F# 5 0.3', 'E 5 0.6', 'B 5 0.4', 'A 5 0.8', 'F# 5 0.8', 'E 5 0.6', 'G 5 0.2', 'F# 5 0.3', 'D# 5 0.7', 'F 5 0.4', 'B 4 1.6', 'B 4 0.3',
'E 5 0.6', 'G 5 0.2', 'F# 5 0.3', 'E 5 0.6', 'B 5 0.4', 'D 6 0.6', 'C# 6 0.3', 'C 6 0.6', 'G# 5 0.3', 'C 5 0.5', 'B 5 0.2', 'A# 5 0.3', 'A# 4 0.6', 'G 5 0.3', 'E 5 1.6',
', 'G 5 0.3', 'B 5 0.6', 'G 5 0.3', 'B 5 0.6', 'G 5 0.3', 'C 6 0.6', 'B 5 0.3', 'A# 5 0.6', 'F# 5 0.3', 'G 5 0.5', 'B 5 0.2', 'A# 5 0.3', 'A# 4 0.6', 'B 4 0.4', 'B 5 1.6',
', 'G 5 0.3', 'B 5 0.7', 'G 5 0.3', 'B 5 0.7', 'G 5 0.3', 'D 6 0.7', 'C# 6 0.3', 'C 6 0.8', 'G# 5 0.3', 'C 6 0.6', 'B 5 0.2', 'A# 5 0.3', 'A# 4 0.6', 'G 5 0.4', 'E 5 1.0',
', 'E 5 1.6']
The arrays match exactly!
```

عکس 5: خروجی کد گام امتیازی